Bringing It Together - Mav’s Ice Cream Emporium (Sprint 4)

Due Tuesday, November 15 at 8 a.m.

CSE 1325 - Fall 2022 - Homework #10 / Sprint 4 - 1

Revision 1

Assignment Background

Welcome back to Java and our class project! I hope you found our C++ side trip invigorating. Now, let's get back to work!

Mav’s Ice Cream Emporium (MICE) is a fledgling start up in the dairy treat market. They are seeking bright young programmers to build a custom solution for defining new confections, tracking customers and the treats they buy, ensuring timely delivery of a quality product, and otherwise taking care of business. Your job is to win this project (with associated profit and glory) from Mav’s Ice Cream Emporium by producing a proposal package over 6 Sprints, including a prototype that wows and other creative and persuasive artifacts that prove you know your stuff.

This is Sprint 4 of 6, in which we add a Container; a Serving that includes a container, some scoops, and some toppings; and an Order that contains 1 or more servings (this is the key class we've been building toward since the start). We'll also get to replace any dialog sequences with slightly more sophisticated dialogs.

The Scrum Spreadsheet

Copy your P07 directory completely to P10. A new Scrum spreadseet is available with several changes to the Product Backlog which should be integrated with your progress to date.

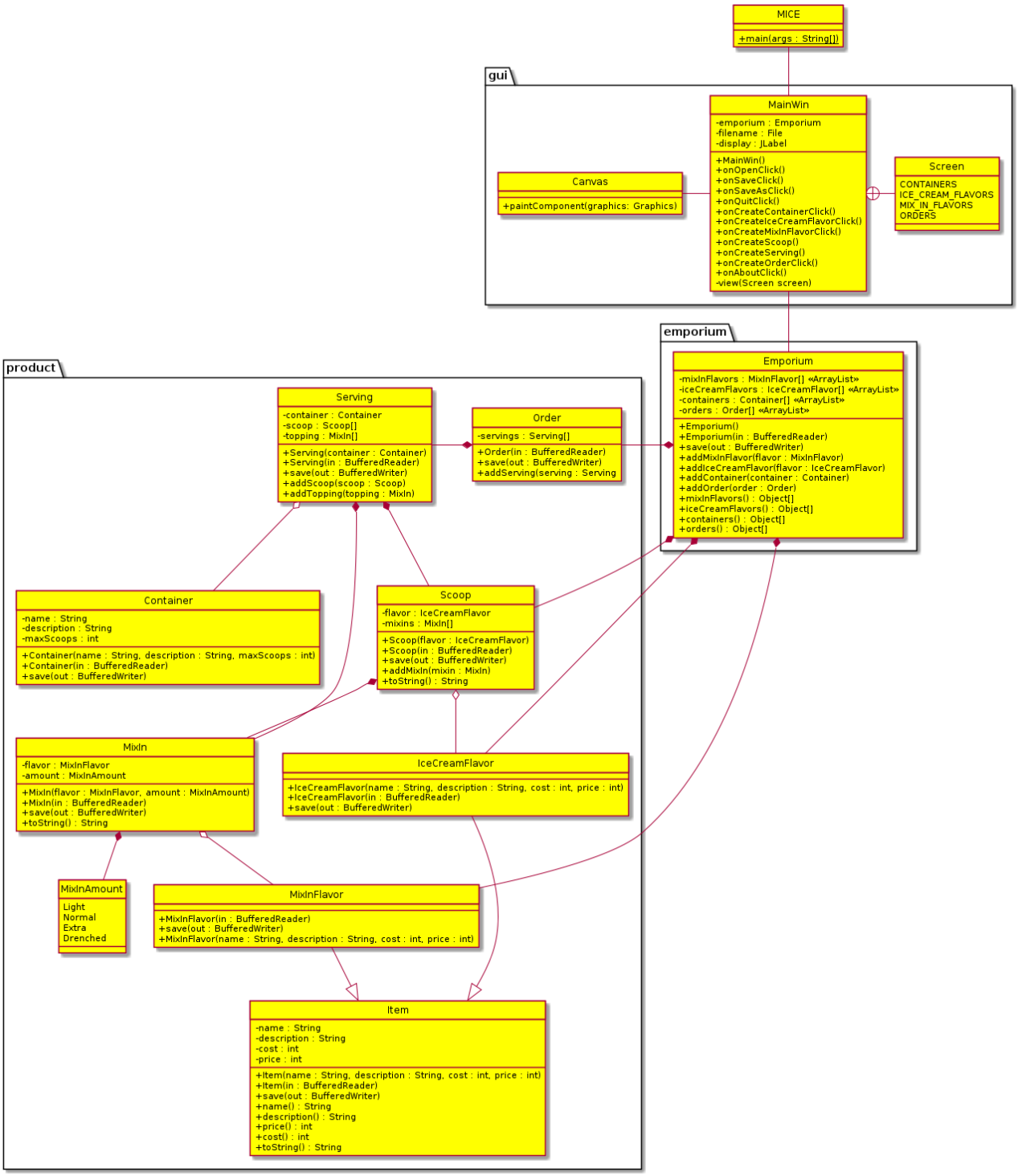
As always, update your Product Backlog tab to reflect your plan and your actual work for Sprint 4. Then, on the Sprint 4 Backlog tab, plan the tasks you'll need to implement the new features (the hints below should help) and update your status as you go.

Be sure to add, commit, and push the updated spreadsheet at cse1325/P10/Scrum_MICE.xlsx before the end of the sprint!

Class Diagram

The diagram on the next page has been updated to refect the new features in this sprint. Note the addition of a save method (with a BufferedWriter parameter) and a constructor (with a BufferedReader parameter) for each class containing data - this is the encapsulation-friendly approach to File > Save and File > Open as we discussed in Lecture 14. Also note the new Canvas class for the fancy About dialog logo. The toolbar feature doesn't affect the class diagram.

As with Sprint 3 you will only be implementing the dark yellow classes this Sprint. Some light guidance follows the class diagram. One or more updates with additional guidance and bug fixes should certainly be expected in any non-trivial project plan such as this.


MICE
+main(args : String[])
gui
MainWin
-emporium : Emporium
-filename : File
-display : JLabel
+MainWin()
+onOpenClick()
+onSaveClick()
+onSaveAsClick()
+onQuitClick()
+onCreateContainerClick()
+onCreateIceCreamFlavorClick()
+onCreateMixInFlavorClick()
+onCreateScoop()
+onCreateServing()
+onCreateOrderClick()
+onAboutClick()
-view(Screen screen)
Canvas
+paintComponent(graphics: Graphics)
Screen
CONTAINERS
ICE_CREAM_FLAVORS
MIX_IN_FLAVORS
ORDERS
emporium
Emporium
-mixInFlavors : MixInFlavor[] «ArrayList»
-iceCreamFlavors : IceCreamFlavor[] «ArrayList»
-containers : Container[] «ArrayList»
-orders : Order[] «ArrayList»
+Emporium()
+Emporium(in : BufferedReader)
+save(out : BufferedWriter)
+addMixInFlavor(flavor : MixInFlavor)
+addIceCreamFlavor(flavor : IceCreamFlavor)
+addContainer(container : Container)
+addOrder(order : Order)
+mixInFlavors() : Object[]
+iceCreamFlavors() : Object[]
+containers() : Object[]
+orders() : Object[]
product
Serving
-container : Container
-scoop : Scoop[]
-topping : MixIn[]
+Serving(container : Container)
+Serving(in : BufferedReader)
+save(out : BufferedWriter)
+addScoop(scoop : Scoop)
+addTopping(topping : MixIn)
Order
-servings : Serving[]
+Order(in : BufferedReader)
+save(out : BufferedWriter)
+addServing(serving : Serving
Container
-name : String
-description : String
-maxScoops : int
+Container(name : String, description : String, maxScoops : int)
+Container(in : BufferedReader)
+save(out : BufferedWriter)
Scoop
-flavor : IceCreamFlavor
-mixins : MixIn[]
+Scoop(flavor : IceCreamFlavor)
+Scoop(in : BufferedReader)
+save(out : BufferedWriter)
+addMixIn(mixin : MixIn)
+toString() : String
MixIn
-flavor : MixInFlavor
-amount : MixInAmount
+MixIn(flavor : MixInFlavor, amount : MixInAmount)
+MixIn(in : BufferedReader)
+save(out : BufferedWriter)
+toString() : String
IceCreamFlavor
+IceCreamFlavor(name : String, description : String, cost : int, price : int)
+IceCreamFlavor(in : BufferedReader)
+save(out : BufferedWriter)
MixInAmount
Light
Normal
Extra
Drenched
MixInFlavor
+MixInFlavor(in : BufferedReader)
+save(out : BufferedWriter)
+MixInFlavor(name : String, description : String, cost : int, price : int)
Item
-name : String
-description : String
-cost : int
-price : int
+Item(name : String, description : String, cost : int, price : int)
+Item(in : BufferedReader)
+save(out : BufferedWriter)
+name() : String
+description() : String
+price() : int
+cost() : int
+toString() : String

Add a Container

(Feature CC) Write the Container class. This is a relatively simple class quite similar to Item. I literally copied and modified Item's code for this class, and recommend you take this approach as well. Don't forget to continue to support saving and loading an Emporium, so you do need that constructor with a BufferedReader parameter and a save method with a BufferedWriter parameter.

Modify class Emporium to maintain an ArrayList of Container objects, along with an addContainer method and a getter that returns an Object[] of Container objects for MainWin to use in a combo box.

Add a Serving

(Feature CS) A Serving (of ice cream) consists of exactly one container, along with 1 or more scoops and 0 or more toppings. (Note that a topping is just a MixIn object, exactly as is used within an ice cream scoop - the MixIn class and array lists are used for both.)

This class is fairly similar to a Scoop in structure, and so I copied and modified that class' code for this class and recommend you do the same (including constructor and save method).

Note that the Emporium doesn't store a list of Serving objects in an ArrayList - they are created on the fly as part of creating an Order. (In Sprint 5 the plan is for each Customer to retain their favorite Servings in a HashMap for faster ordering. We just happen to cover the Java Class Library with HashMaps next week. What a coincidence.)

In Sprint 3, this class was shown with two methods for calculating the price and cost of a Serving. These have been moved to Sprint 5, which will focus on the Customer interactions with the store. Most of the features originally planned for Sprint 5 will become bonus features.

Add an Order

(Feature CO) An Order consists of one or more Servings and (in Sprint 5) will generate a receipt. It's what really matters. As with Serving, this class is fairly similar to a Scoop in structure, and so I also copied and modified that class' code for this class to save time (including constructor and save method).

A list of Order objects will be stored in the Emporium and displayed by MainWin. This will *replace* the storing of Scoop objects - Scoops are now just a part of Servings, which are just part of Orders, so all 3 objects are effectively stored in the orders ArrayList.

Update the User Interface

(Features CC, CS, and CO) Remove the Create > Scoop menu item and button, and replace with Create > Order. This will invoke onCreateOrderClick().

Also add a Create > Container menu item and button that invokes onCreateContainerClick().

Write onCreateContainerClick()

This dialog is very similar to onCreateMixInFlavorClick(), so I copied that method and modified it to create a new Container object and pass it to Emporium via its addContainer method.

Remove SCOOPS from the Screen enum and add CONTAINERS (if you have one - or do the equivalent with your design). Set the main window display area to show a list of Container objects when this method completes.

Modify onCreateScoopClick()

Rather than adding the Scoop object via Emporium.addScoop (which should be removed), return the Scoop directly. You no longer need to set the display to list scoops (which are no longer separately tracked), and if you have a setDirty() method it's not needed here either. (You may choose to remove the word "Click" from the method name, since this now becomes a utility method rather than a Listener. Up to you.)

Write onCreateServingClick()

This method is quite similar to onCreateScoopClick(), except that the user must select a Container rather than an IceCreamFlavor for the constructor, then one or more Scoops (created using the onCreateScoopClick() method), and finally zero or more toppings (using the list of MixIns as onCreateScoopClick() did).

As with the new onCreateScoopClick() method, return the actual Serving created. Emporium doesn't care - it's not keeping a list of Serving objects.

Write onCreateOrderClick()

This is the key method - we make money by selling Orders!

This method is similar to onCreateScoopClick() but simpler - we merely need to create and add one or more Servings using onCreateServingClick() and then add the Order to Emporium via its new addOrder method. Also add ORDERS to the Screen enum (or equivalent in your design) and set the view to a list of Order objects in your main window.

Hints for Adding 3 Classes

package product

The existing classes in this package are unlikely to require any changes.

The three new classes - Container, Serving, and Order - are unremarkable and should not present a big challenge.

package emporium

The Emporium class must be updated to *remove* managing Scoop objects (which are now just a field in a Serving object) and to *add* managing Container and Order objects. The following changes are expected:

- Add imports for Container and Order, and remove the import of Scoop.
- Add ArrayLists for Container and Order, and remove the ArrayList for Scoop.
- Add methods addContainer and addOrder, add getters containers and orders, and remove addScoop and scoops.
- Update method save to replace saving the scoops ArrayList with saving the containers and orders ArrayLists.
- Update the constructor to replace loading the scoops ArrayList with loading the containers and orders ArrayLists.

package gui

You will need button icons for adding a container and an order. The suggested solution reused the image from add scoop for add order - close enough, I think.

The MainWin class must be updated to replace Create > Scoop with Container and Order, with similar changes to the View menu. The toolbar must be similarly updated.

onCreateContainerClick()

This method is very similar to onCreateIceCreamFlavor(), so copying and modifying that code may save time.

onCreateScoop()

(Formerly onCreateScoopClick()) Since Emporium is no longer managing a list of Scoop objects, this method will now be called when a new Scoop is needed from the customer (for example, when creating a Serving). Thus, the method should return a new Scoop object, or null if the customer clicked Cancel.

onCreateServing()

As with onCreateScoop(), this method is called when a new Serving is needed (for example, when creating an Order).

- First, obtain a Container selected by the user in a JOptionPane dialog. Use this to instance a new Serving.
- Second, obtain one or more Scoops from the user, each in a JOptionPane dialog. If Cancel is clicked for the first Scoop, discard the Serving object and return a null. For each Scoop selected, add it to the Serving instance.
- Finally, obtain zero or more toppings from the user, each in a JOptionPane dialog. (A topping is a MixIn instance, using exactly the same list from Emporium for the MixInFlavor and a topping amount via the MixInAmount enum.) For each topping selected, add it to the Serving instance.
- Once all Scoops and toppings have been added, return the Serving object.

A try/catch isn't a bad idea here. The catch can either display an error dialog or log the error to Standerd.err or a file.

onCreateOrderClick()

This is a simpler but similar implementation to the onCreateServing() method. It is void, because the new Order will be added to Emporium rather than returned.

The default constructor for Order has no parameter, so the Order can be created immediately.

- Loop, calling onCreateServing() and (if not null) adding the Serving to the Order instance. Exit the loop when a null is returned.
- If at least one Serving was added to the Order, pass it to Emporium.addOrder. Otherwise, discard the Order object and return.

view

The view(Screen) method must be updated to remove code for showing Scoops, and to add code for showing Containers and Orders in the main window as requested.

So 4 different displays are required for this sprint:

- Containers
- Ice cream flavors
- Mix in flavors
- Orders

The suggested solution also shows the order number (that is, the index of the Order object in the ArrayList), but this isn't required.

Hints for Unified Dialogs

The dialogs that should be unified in this sprint are

- Create New Container
- Create New Ice Cream Flavor
- Create New MixIn Flavor
- Create New MixIn (this gets the MixInFlavor and the MixInAmount from two combo boxes)

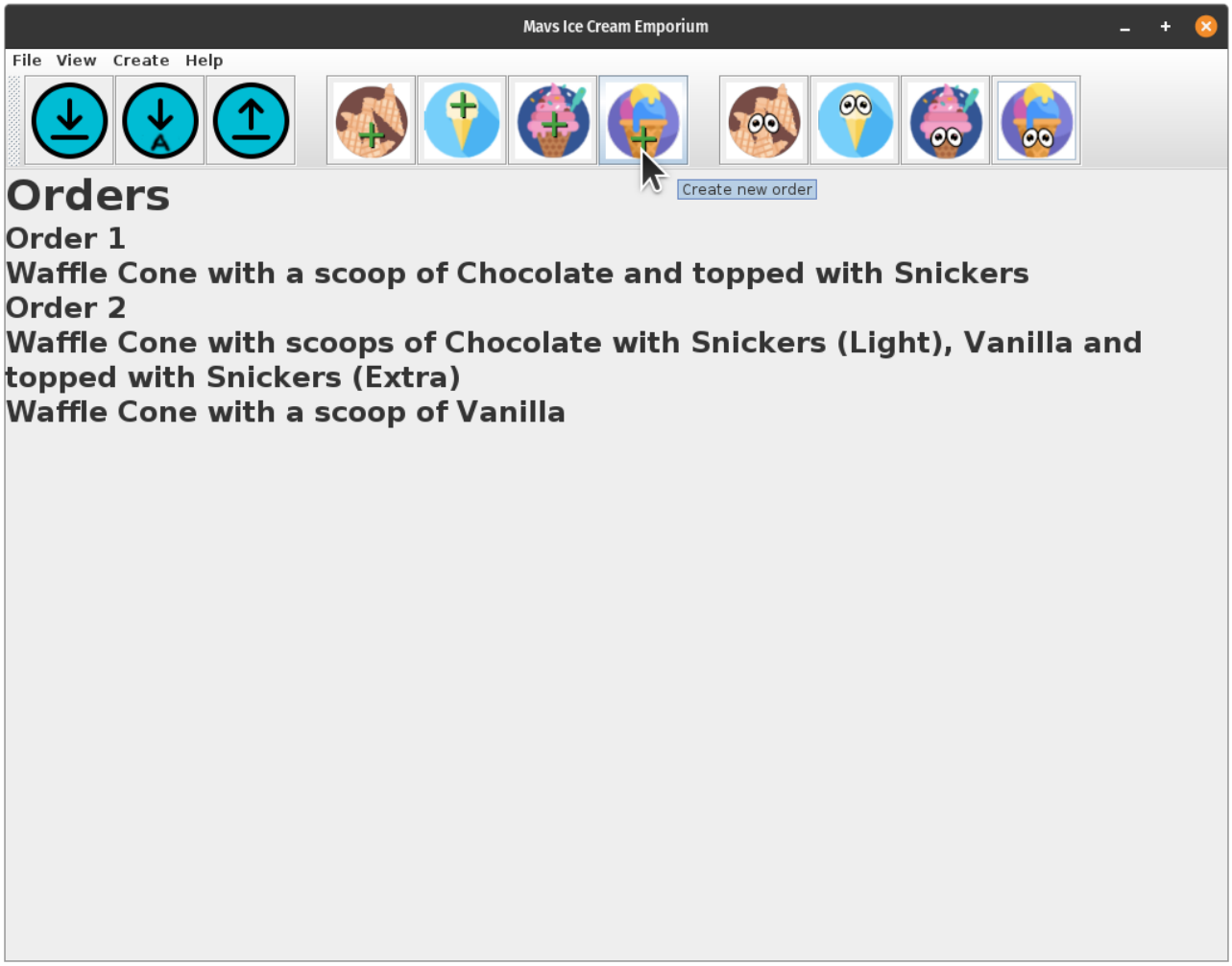
It may also be possible to create more sophisticated unified dialogs to create Scoop, Serving, and Order objects, but those are NOT required.

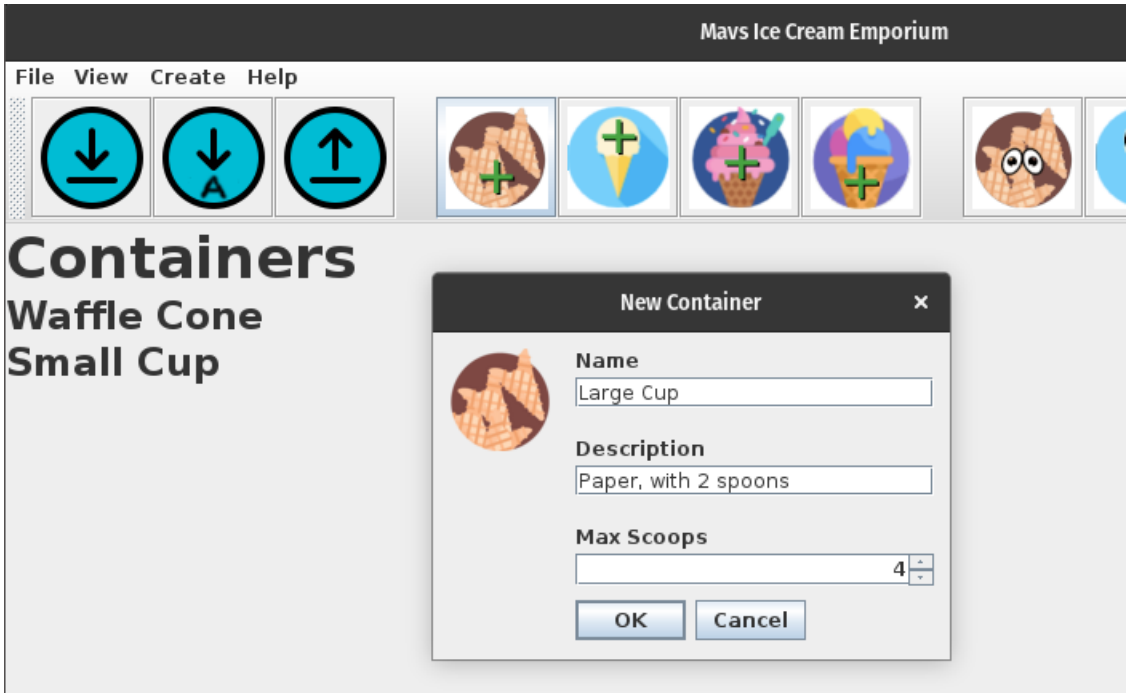
Lecture 13 has extensive discussion on how to create unified dialogs. A good example is the onButtonClick() method in cse1325-prof/13/code_from_slides/AnimalJOptionPane.java.

Screenshots

Your application is NOT required to look like the suggested solution! You have significant freedom now to make this project your own, subject only to meeting the stated requirements. If you have questions as to whether your plans conform to the requirements, I'm happy to discuss with you via email or in my office.

Screenshots will be added here in the first update.


Mavs Ice Cream Emporium
File View Create Help
Create new order
Orders
Order 1
Waffle Cone with a scoop of Chocolate and topped with Snickers
Order 2
Waffle Cone with scoops of Chocolate with Snickers (Light), Vanilla and topped with Snickers (Extra)
Waffle Cone with a scoop of Vanilla


Mavs Ice Cream Emporium
File
View
Create
Help
Containers
Waffle Cone
Small Cup
New Container
Name
Large Cup
Description
Paper, with 2 spoons
Max Scoops
4
OK
Cancel